Robot Operating Systems

Robot Software Development

Abdelbacet Mhamdi

2025-11-20

MT @ ISET Bizerte

Outline

1. ROS 2 Foundations

TurtleSim is a beginner-friendly tool for learning ROS 2 concepts through visual, interactive examples. This lightweight simulator offers a 2D environment where we control virtual turtles that can move around, draw lines, and respond to commands.

1.1.1 Installation

TurtleSim comes pre-installed with the ROS 2 desktop installation. We can also install it using the following command:

```bash
sudo apt install ros-humble-turtlesim # For Ubuntu 22: ROS 2 Humble
```

The way to verify the installation is by checking the available executables:

```bash
ros2 pkg executables turtlesim
```

1.1.2 Launching TurtleSim

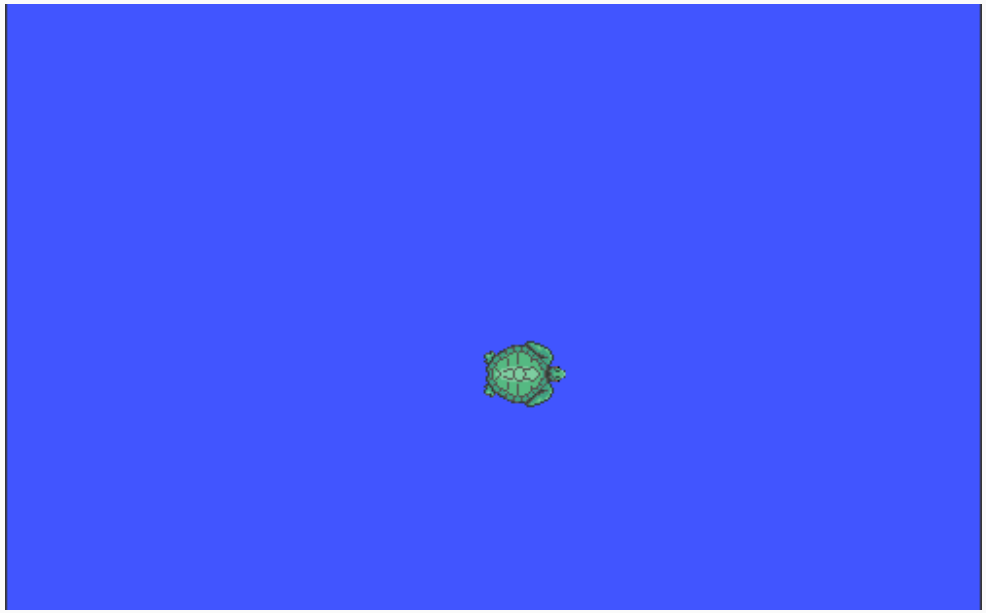
The first step is launching the *turtlesim_node*, which creates the simulation window:

```bash
ros2 run turtlesim turtlesim_node
```

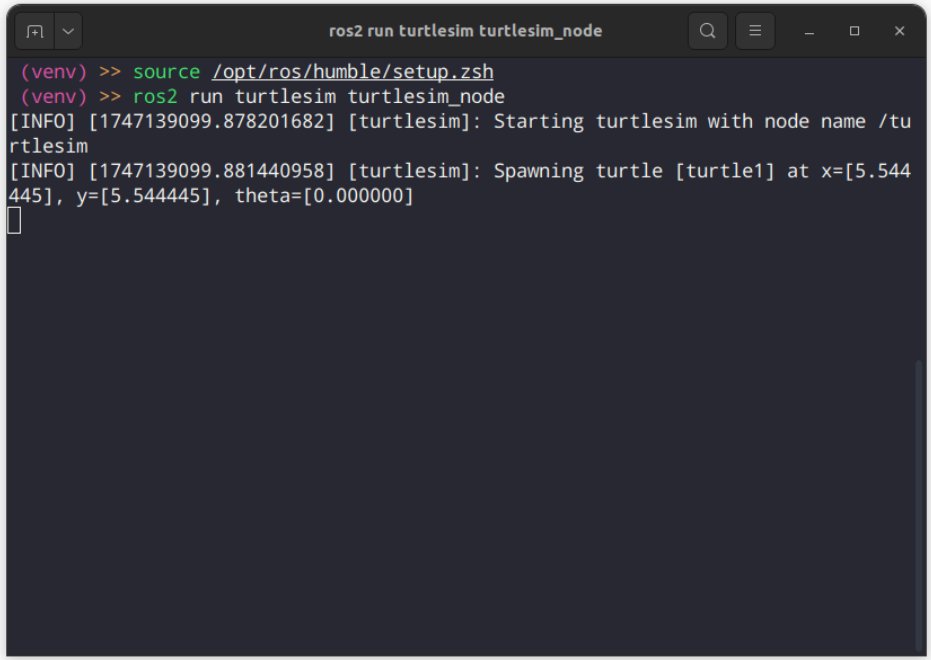
This command starts a blue simulation window with a turtle in the center.

This node:

- creates the simulation environment
- manages turtle state (position, orientation, pen status)
- processes incoming commands
- publishes turtle pose information



- The turtle's position and orientation are represented in a 2D coordinate system.
- The turtle can move forward, backward, and rotate, and its pen can be raised or lowered to draw on the canvas.

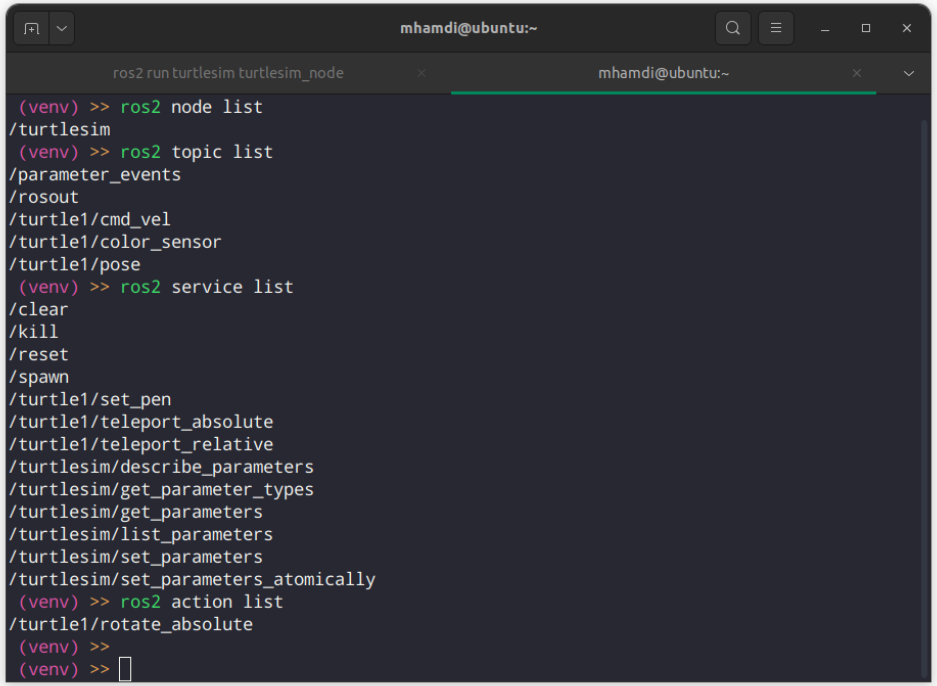


1.1.2.1 List Active Components

The turtle's state is managed by the *TurtleSim* node, which processes incoming commands and publishes the turtle's pose information. This allows us to control the turtle's movement and visualize its position in real time.

Bash

```bash
# running nodes
ros2 node list
# active topics
ros2 topic list
# available services
ros2 service list
# display actions
ros2 action list
```



1.1.2.2 Moving the Turtle

```bash
ros2 run turtlesim turtle_teleop_key
```

Info

Keep the teleop terminal window focused while sending keyboard commands.

Memorize

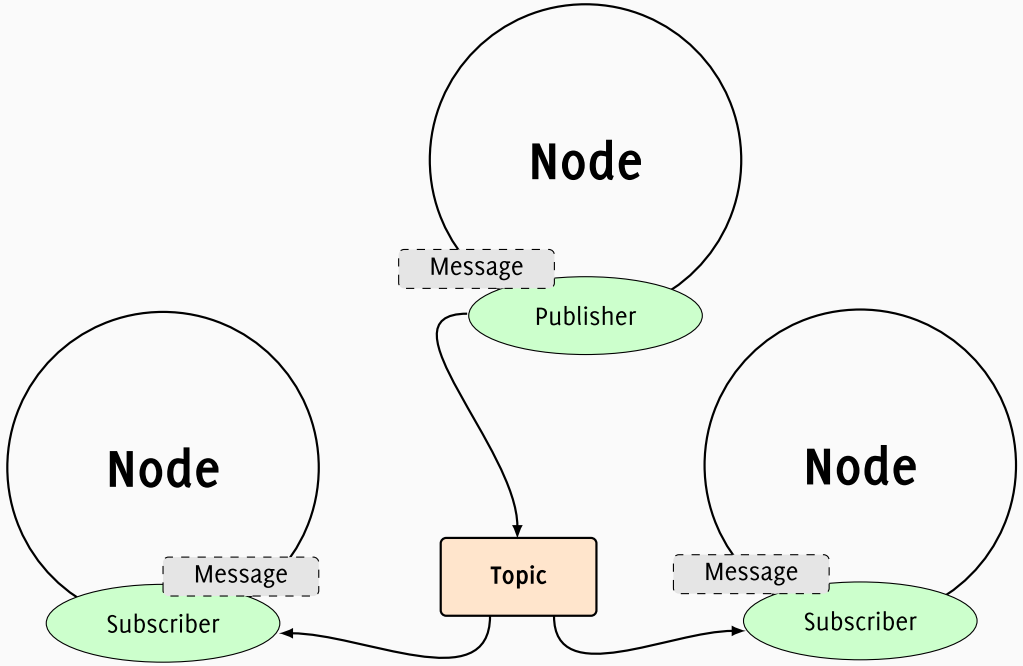
Arrow keys Move the turtle forward/backward and rotate left/right

Space Stop the turtle

1.1.3 Programmatic Control

1.1.3.1 Using Topics

Topics are named communication channels for message passing.



We can explore active topics via:

```bash
# List all active topics
ros2 topic list
```

Key topics include:

/turtle1/cmd_vel For sending velocity commands

/turtle1/color_sensor Color detected by the turtle

/turtle1/pose Published turtle position and orientation

Examine topic types and message structures:

```bash
ros2 topic type /turtle1/cmd_vel # Result: geometry_msgs/msg/Twist
```

```bash
ros2 topic echo /turtle1/pose # Echo messages on a topic
```

1. **Forward**

```bash
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear:
{x: 2., y: 0., z: 0.}, angular: {x: 0., y: 0., z: 0.}}"
```

2. **Backward**

```bash
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear:
{x: -2., y: 0., z: 0.}, angular: {x: 0., y: 0., z: 0.}}"
```

3. **Rotate Left**

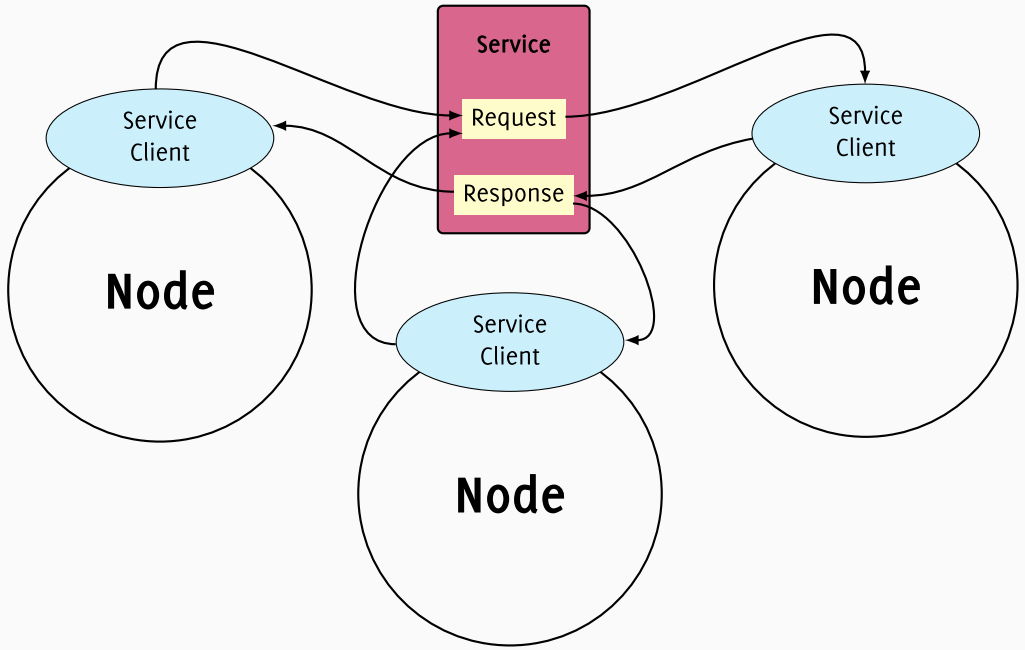
```bash
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear:
{x: 0., y: 0., z: 0.}, angular: {x: 0., y: 0., z: 1.}}"
```

4. **Rotate right**

```bash
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear:
{x: 0., y: 0., z: 0.}, angular: {x: 0., y: 0., z: -1.}}"
```

1.1.3.2 Using Services

Services enable us to request specific actions or information from the turtle's services.



To list all active services, we use:

```bash
ros2 service list -t
```

1. **Set Absolute Position**

```bash
ros2 service call /turtle1/teleport_absolute turtlesim/srv/TeleportAbsolute "{x: 5., y: 5., theta: 0.}"
```

2. **Set Relative Position**

```bash
ros2 service call /turtle1/teleport_relative turtlesim/srv/TeleportRelative "{linear: 1., angular: 1.}"
```

3. **Changing the Turtle's Color**

```bash
ros2 service call /turtle1/set_pen turtlesim/srv/SetPen "{r: 255, g: 0, b: 0, width: 2, 'off': 0}"
```

4. **Clearing the Screen**

```bash
ros2 service call /clear std_srvs/srv/Empty "{}"
```

5. **Resetting the Turtle**

```bash
ros2 service call /reset std_srvs/srv/Empty "{}"
```

1.2 Turtle Drawing Challenge

Use `turtlesim` to create a square pattern using only ROS 2 command-line tools. The goal is to make the turtle draw a perfect square without any programming.

1.2.1 What to Observe

- How does the turtle's movement affect the drawing?
- What happens when you change the pen color mid-drawing?
- How precise can you make the square using only command-line tools?

1.2.2 Learning Objectives

- Understand ROS 2 topic publishing
- Learn about service calls
- Practice with `geometry_msgs/msg/Twist` messages
- Experience real-time robot control concepts

Task 1: Pen Control

Before starting the square, set the pen to:

- Red color (R=255, G=0, B=0)
- Width of 3 pixels
- Pen down (off=0)

```bash
# Set pen to red color and width 3
ros2 service call /turtle1/set_pen turtlesim/srv/SetPen "{r: 255, g: 0, b: 0, width: 3, 'off': 0}"
```

Task 2: Basic Movement

Use the `ros2 topic pub` command to make the turtle:

- Move forward 2 units
- Rotate 90 degrees to the right
- Move forward 2 units
- Rotate 90 degrees to the right
- Move forward 2 units
- Rotate 90 degrees to the right
- Move forward 2 units
- Rotate 90 degrees to the right

Tip

Use `geometry_msgs/msg/Twist` messages with:

- `linear.x` for forward/backward movement
- `angular.z` for rotation (positive = left, negative = right)

Info

Don't forget to reset the turtle's position if needed:

```bash
ros2 service call /reset std_srvs/srv/Empty "{}"
```

```bash
# Move forward 2 units
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}}"

# Rotate 90 degrees right
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{angular: {z: -1.57}}"
```

We could also create a square using `/turtle1/teleport_absolute` service:

Bash

```bash
# Teleport to corners of square
ros2 service call /turtle1/teleport_absolute turtlesim/srv/TeleportAbsolute "{x: 7.5, y: 5.5}" && sleep 1

ros2 service call /turtle1/teleport_absolute turtlesim/srv/TeleportAbsolute "{x: 7.5, y: 3.5}" && sleep 1

ros2 service call /turtle1/teleport_absolute turtlesim/srv/TeleportAbsolute "{x: 5.5, y: 3.5}" && sleep 1

ros2 service call /turtle1/teleport_absolute turtlesim/srv/TeleportAbsolute "{x: 5.5, y: 5.5}"
```

Using `/turtle1/teleport_relative` service is also possible:

Bash

```
ros2 service call /turtle1/teleport_relative turtlesim/srv/
TeleportRelative "{linear: 2.}" && sleep 1

ros2 service call /turtle1/teleport_relative turtlesim/srv/
TeleportRelative "{linear: -2., angular: 1.57}" && sleep 1

ros2 service call /turtle1/teleport_relative turtlesim/srv/
TeleportRelative "{linear: 2., angular: 1.57}" && sleep 1

ros2 service call /turtle1/teleport_relative turtlesim/srv/
TeleportRelative "{linear: 2., angular: -1.57}"
```

Task 3: Precision Challenge

Try to make the turtle return to its starting position after completing the square. Use the `teleport_absolute` service to verify the final position.

To check if the turtle returned to its starting position:

Bash

```
# Check current position
ros2 topic echo /turtle1/pose

# If needed, teleport to exact starting position
ros2 service call /turtle1/teleport_absolute turtlesim/srv/TeleportAbsolute "{x: 5.5, y: 5.5, theta: 0.0}"
```

1.2.3 Summary and Key Takeaways

Understanding Twist Messages

- `linear.x`: Forward/backward velocity (positive = forward)
- `angular.z`: Rotational velocity (positive = counterclockwise, negative = clockwise)
- Values are in radians per second for rotation

Timing Considerations

- The `sleep` commands are crucial for allowing the turtle to complete each movement
- Without proper timing, movements overlap and the pattern becomes distorted
- Real robots would use feedback control instead of fixed timing

Service vs Topic Usage

- **Topics**: For continuous commands (velocity control)
- **Services**: For discrete actions (teleportation, pen control)

Warning

- If the turtle doesn't move, check that the Turtlesim window is active
- If movements are too fast/slow, adjust the sleep duration
- If the square isn't closed, verify the rotation angles ($90° = \frac{\pi}{2} \approx 1.57$ radians)
- Use `ros2 topic echo /turtle1/pose` to monitor the turtle's position in real-time

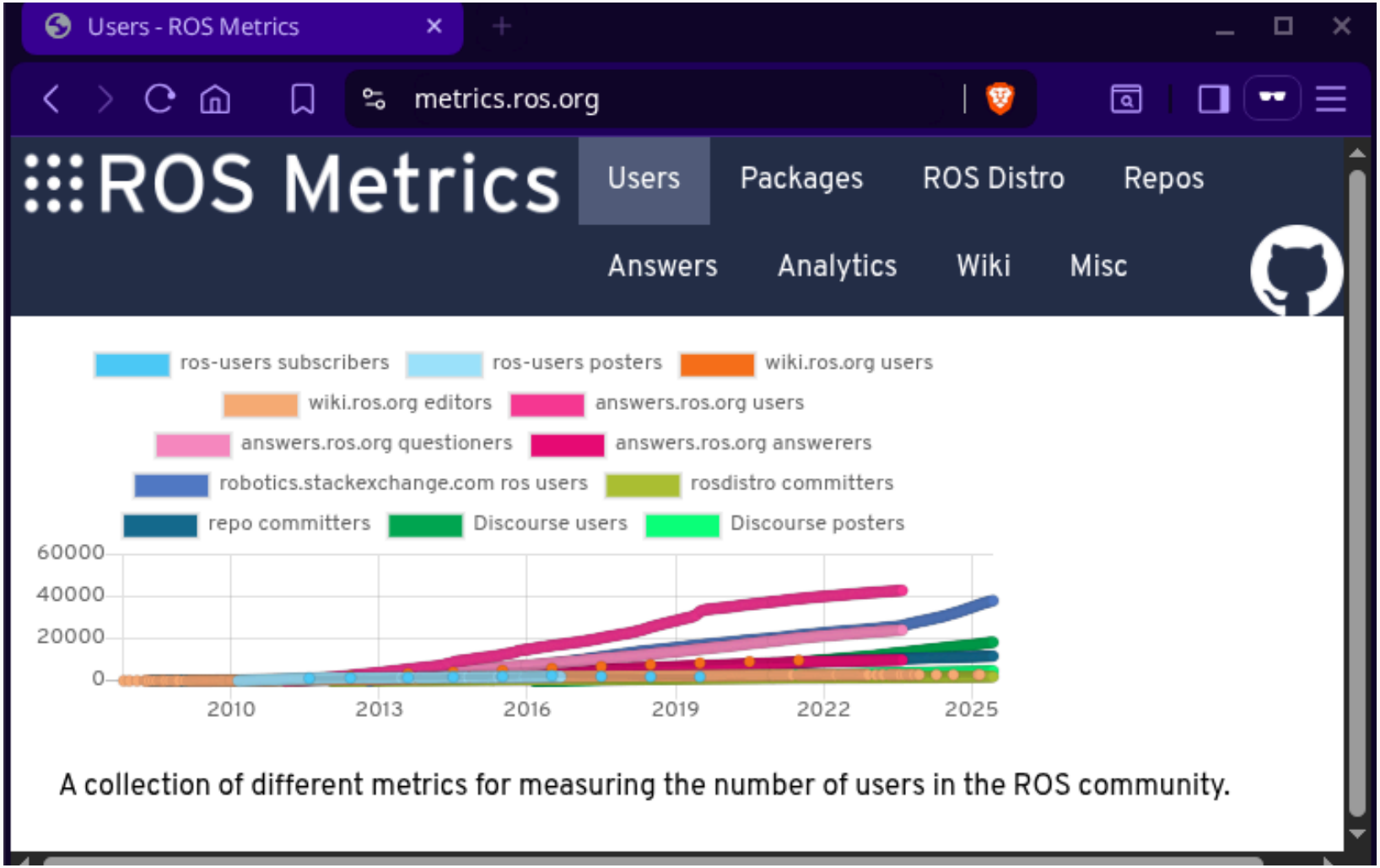
Idea

- Try creating different shapes (triangle, pentagon, hexagon)
- Experiment with different pen colors and widths
- Create patterns by combining multiple shapes
- Use relative teleportation for more complex movements

The **Robot Operating System** (**ROS**) is an open-source, flexible framework for writing robot software. It is not an operating system in the traditional sense but rather a collection of software libraries, tools, and conventions that aim to simplify the task of creating complex and robust robot behavior across a wide variety of robotic platforms.

ROS provides functionalities typically expected from an **OS**, including hardware abstraction, low-level device control, implementation of commonly-used functionality, message-passing between processes, and package management.

> 🏁 **Goal**
>
> Its primary goal is to support code reuse in robotics research and development.



A collection of different metrics for measuring the number of users in the ROS community.

Modular Architecture

ROS is built on a distributed computing model, allowing developers to create modular software packages that can easily communicate with each other. This approach enables:

- Flexible robot software design
- Easy integration of different components
- Reusability of code across different robotic projects

Communication Infrastructure

At its core, **ROS** provides a robust communication infrastructure that allows different parts of a robotic system to exchange information:

- Publish-subscribe messaging model
- Service-client communication
- Action servers for long-running tasks
- Support for multiple programming languages (primarily C++ and Python)

Extensive Toolset

ROS offers a comprehensive set of tools for various aspects of robotics development:

- Simulation environments (like **Gazebo**)
- Visualization tools (like **RViz**)
- Debugging and logging utilities
- Hardware abstraction layers
- Package management system

Applications and Usage

ROS is widely adopted in:

- Academic research institutions
- Robotics laboratories
- Industrial robotics
- Autonomous vehicle development
- Research and development in various domains (manufacturing, healthcare, aerospace)

Danger

Limitations of **ROS 1**

While **ROS 1** achieved significant success, the evolving robotics landscape highlighted several limitations in its design, including:

Multi-robot systems Struggled with decentralized communication and discovery in large or dynamic networks.

Real-time control Lacked native support for real-time performance requirements.

Production environments Offered limited security features, making it less suitable for commercial deployments.

Resource-constrained platforms Proved resource-heavy for embedded systems.

Network reliability Depended on a single master node, introducing a single point of failure.

Success

ROS 2 Overview

ROS 2 was designed from scratch to overcome the limitations of ROS 1, delivering a more robust, secure, and adaptable platform. It supports a wide range of applications, from research prototypes to industrial and commercial systems.

Enhanced Multi-Robot Support Enables decentralized communication and discovery, addressing **ROS 1**'s challenges with dynamic, large-scale robot networks.

Real-Time Capabilities Provides native support for real-time control, making it suitable for time-critical robotic applications.

Improved Security Incorporates robust security features to meet the needs of commercial and industrial deployments.

Resource Efficiency Optimized for resource-constrained platforms, such as embedded systems, unlike the heavier **ROS 1**.

Network Reliability Eliminates the single point of failure by removing dependency on a single master node, improving system resilience.

Distro	Logo	Release	EOL
Kilted Kaiju		23 mai 2025	December 2026
Jazzy Jalisco		23 mai 2024	May 2029
Iron Irwini		23 mai 2023	November 2024
Humble Hawksbill		23 mai 2022	mai 2027
Galactic Geochelone		23 mai 2021	9 december 2022

Foxy Fitzroy		5 juin 2020	20 june 2023
Eloquent Elusor		22 novembre 2019	novembre 2020
Dashing Diademata		31 mai 2019	mai 2021
Crystal Clemmys		14 december 2018	december 2019
Bouncy Bolson		2 july 2018	july 2019
Ardent Apalone		8 december 2017	december 2018

Notification

Introducing ROS 2 Humble Hawksbill

ROS 2 Humble Hawksbill, the eighth major release of **ROS 2**, was officially launched on May 23, 2022. It marks a key milestone in the **ROS 2** ecosystem, enhancing previous versions with new features and improvements.

- **Humble Hawksbill** is an LTS release, offering stability and extended maintenance for developers.
- Support includes bug fixes and security patches until May 2027.
- Its long support period suits commercial products, extended research projects, and educational use.
- Primarily targets **Ubuntu 22.04 Jammy Jellyfish** (`amd64` and `arm64` architectures).
- Also compatible with **Windows 10**.

Improvements in Humble

- ROSBag Enhancements
- Performance and Stability Gains
- Enhanced Documentation
- Developer Ergonomics
- Gazebo Fortress Integration
- FogROS2
- Foxglove Studio

2. Nodes and Communication

A **ROS 2** node is a fundamental computational unit in the Robot Operating System 2 that represents an executable process within a robotic system.

2.1.1 Core Components

- Written in C++ or Python
- Uses `rclcpp` (C++) or `rclpy` (Python) client libraries
- Supports multiple communication patterns

2.1.2 Types of Nodes

2.1.2.1 Sensor Nodes

- Collect and process sensor data

Example

- Camera node
- LIDAR node
- IMU sensor node

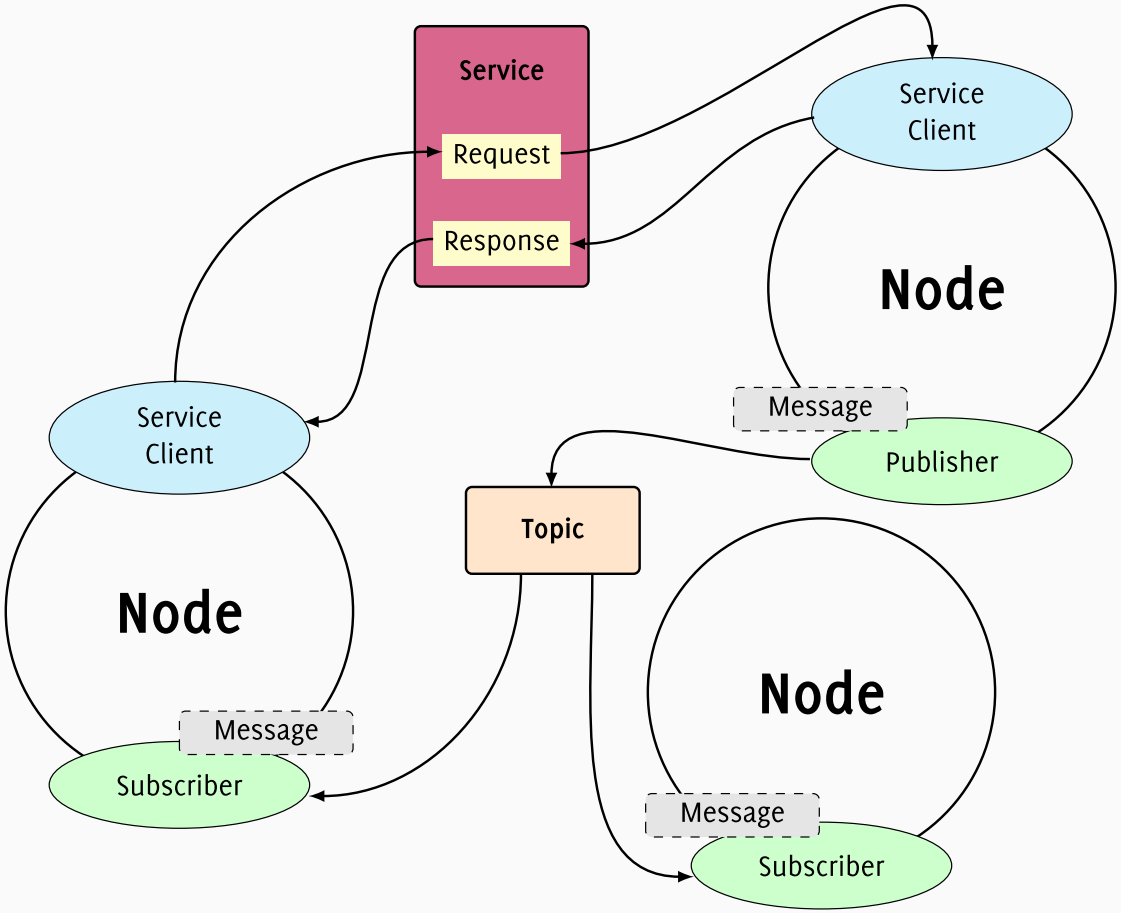
2.1.2.2 Actuator Nodes

- Control robotic actuators
- Manage precise motor movements

2.1.2.3 Processing Nodes

- Implement computational algorithms
- Perform data analysis and decision-making

2.1 What is a ROS 2 Node?



```bash
# Launch individual nodes
ros2 run <pkg_name> <node_name>
# Launch multiple nodes with configuration
ros2 launch <pkg_name> <launch_file>
# Display active nodes
ros2 node list
# Show detailed information about a specific node
ros2 node info <node_name>
```

Topics, services, and actions are core communication mechanisms used to enable nodes to exchange data, request computations, or manage long-running tasks.

2.2.1 Topics

- Provide a publish-subscribe communication model
- Allow nodes to send messages to multiple subscribers
- Useful for broadcasting information

```bash
# Publisher
ros2 topic pub /chatter std_msgs/msg/String "data: 'Hello, ROS 2!'"
# Subscriber
ros2 topic echo /chatter
```

2.2.2 Services

- Provide a request-response communication model
- Allow nodes to call functions on other nodes
- Useful for tasks that require a single response

```bash
# Server
ros2 run demo_nodes_cpp add_two_ints_server
# Client
ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a: 5, b: 3}"
```

2.2.3 Actions

- Designed for long-running tasks
- Allow feedback during execution
- Support preemption and cancellation

Task 4: Theory

1. What is a ROS 2 node?
2. What is the difference between a topic and a service in ROS 2?
3. Give an example use case for a topic and one for a service.

1. A **ROS 2** node is a fundamental process that performs computation in a **ROS 2** system. Each node is an independent executable that can communicate with other nodes using topics, services, and actions.
2. A topic in **ROS 2** is used for unidirectional, asynchronous, many-to-many communication (publish/subscribe), suitable for streaming data like sensor readings. A service is used for synchronous, two-way communication (request/response), suitable for tasks that require a reply, like resetting a simulation.
3. **Example use case for a topic** Publishing sensor data from a LIDAR to be consumed by multiple nodes.

Example use case for a service A node requests another node to reset the simulation environment and waits for confirmation.

Task 5: Practical

4. Write the command to list all active nodes in a running ROS 2 system.
5. Suppose you have a node publishing to the topic `/chatter`. Write the command to echo messages from this topic.
6. What is the command to call a service named `/reset_simulation` of type `std_srvs/srv/Empty`?

4. Command to list all active nodes:

```bash
ros2 node list
```

5. Command to echo messages from the `/chatter` topic:

```bash
ros2 topic echo /chatter
```

6. Command to call the `/reset_simulation` service of type `std_srvs/srv/Empty`:

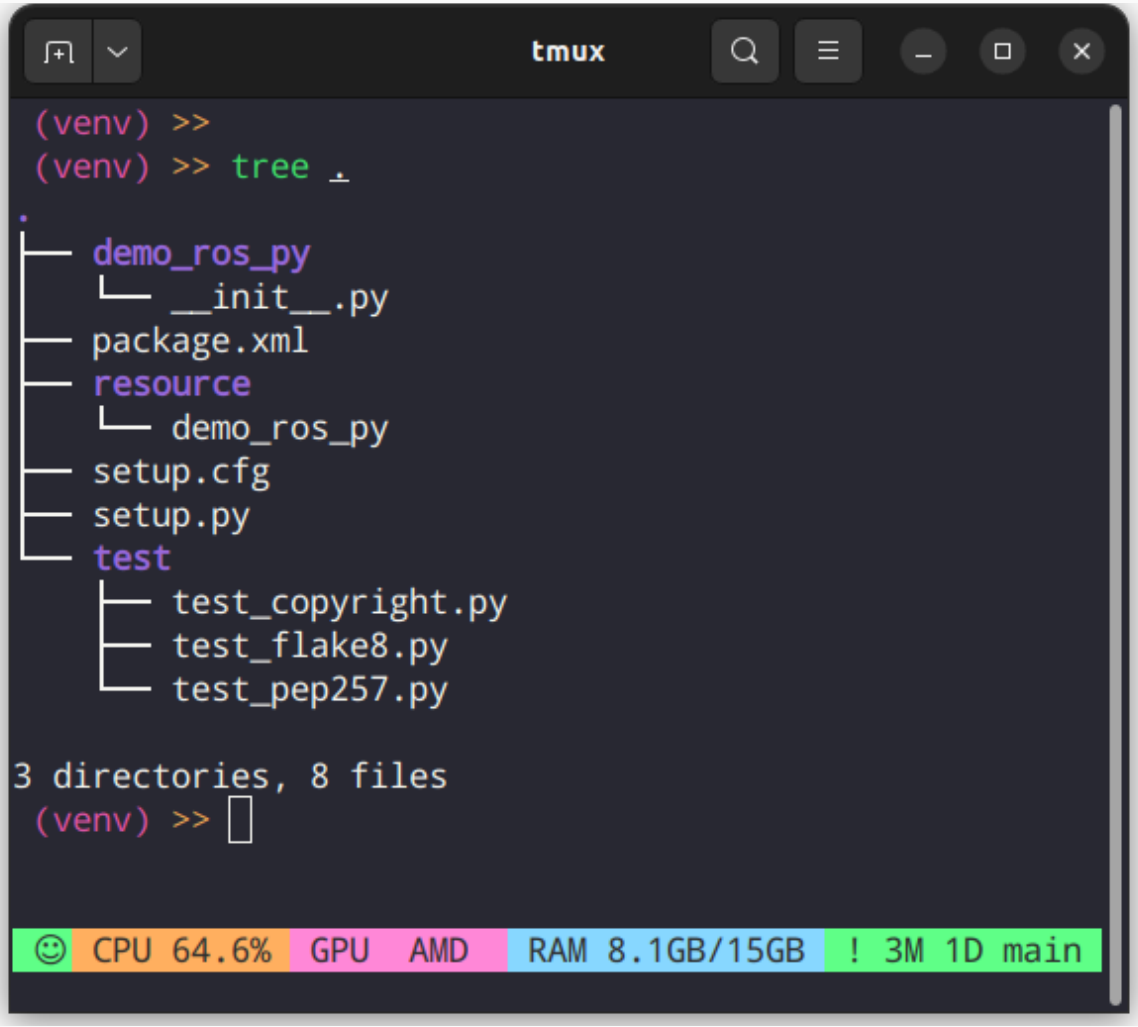
```bash
ros2 service call /reset_simulation std_srvs/srv/Empty "{}"
```

```bash
ros2 pkg create --build-type ament_python <package_name>
```

This command creates a new **ROS 2** package with the specified name, using the `ament_python` build type. The generated package structure will look like this:

```
<package_name>/
├── package.xml
├── setup.py
├── setup.cfg
├── resource/
│   └── <package_name>
├── test/
│   ├── test_copyright.py
│   ├── test_flake8.py
│   └── test_pep257.py
└── <package_name>/
    └── __init__.py
```

2.3 Build mechanism



Info

Root Level Files

`package.xml` The package manifest file containing metadata about the package *(dependencies, version, description, maintainer info, etc.)*

`setup.py` Python setup script that defines how the package should be built and installed

`setup.cfg` Configuration file for setup tools, typically contains console script entry points

The `package.xml` file is a package manifest for **ROS 2** that describes the package. It's written in **XML** and includes key information like:

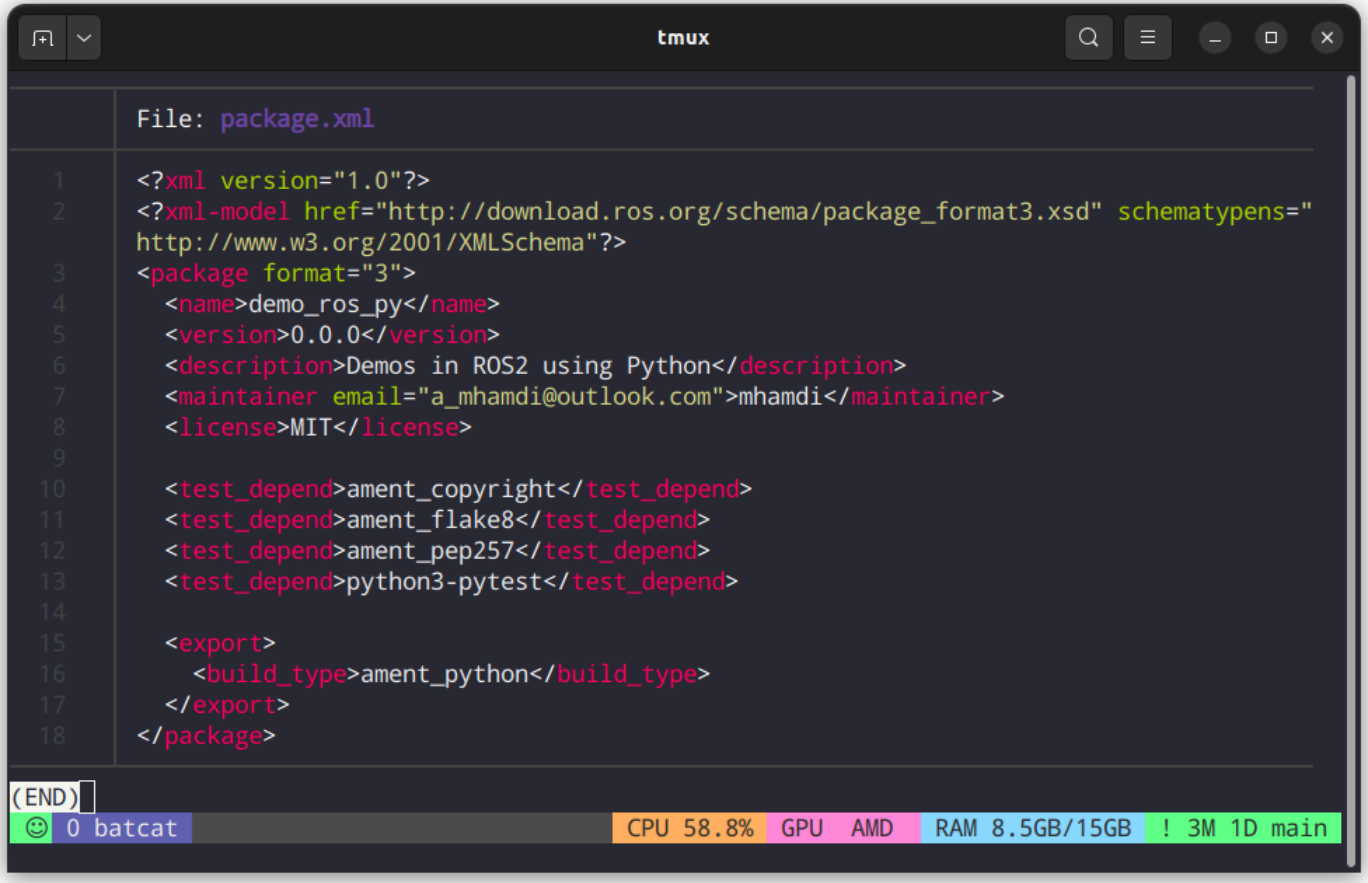
- **Metadata:** The package's name, version, a description, maintainer information, and license.
- **Dependencies:** Other packages required for the current package to build and run.
- **Build System Info:** Details on the build type, such as `ament_python`.
- **Export Tags:** Extra information for the **ROS 2** build system.

Essentially, this file is how **ROS 2** manages dependencies and compiles our package.

To install the required dependencies, we need to navigate to the package directory and run:

```bash
rosdep install -i --from-path src/<package_name> --rosdistro humble -y
```

2.3 Build mechanism

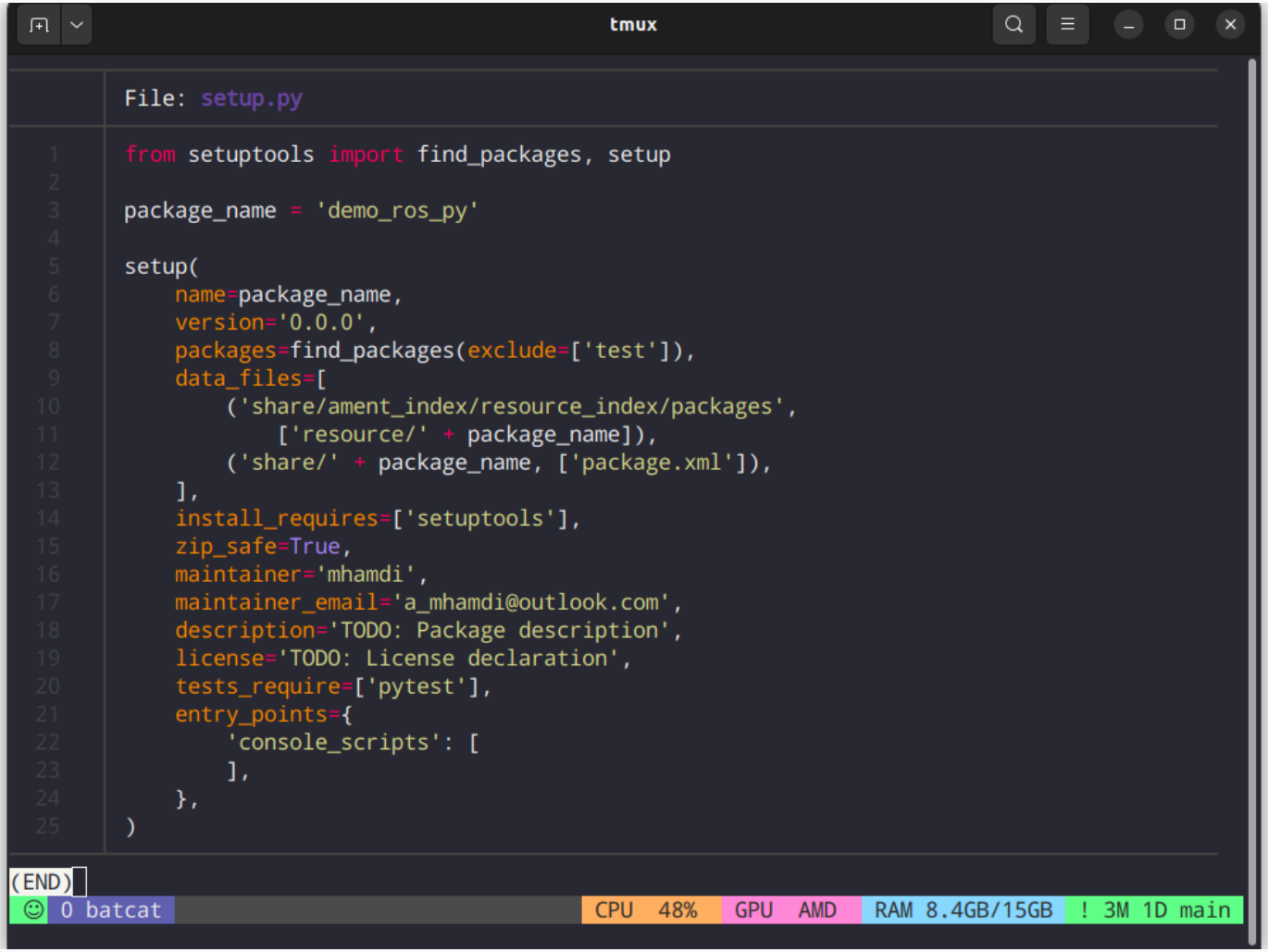


The `setup.py` file is a **Python** script that provides instructions for installing a **ROS 2** package. It includes:

- **Metadata:** Package details such as the name, version, and author, which are often sourced from package.xml.
- **Dependencies:** The required Python packages for the project.
- **Entry Points:** Specifies console scripts that define **ROS 2** nodes, allowing them to be run as executable commands.
- **Data Files:** Information on any extra files, like launch files or configurations, that need to be installed.
- **Package Discovery:** Instructions for setuptools on which **Python** packages to include.

This file uses a standard **Python** packaging mechanism to work with the ament build system, making it possible to install and run our **Python** nodes as **ROS 2** executables.

2.3 Build mechanism



Info

Directories

resource/<package_name> Contains a marker file *(usually empty)* that helps **ROS 2** identify this as a package

test/ Contains basic test files:

test_copyright.py Checks for proper copyright headers

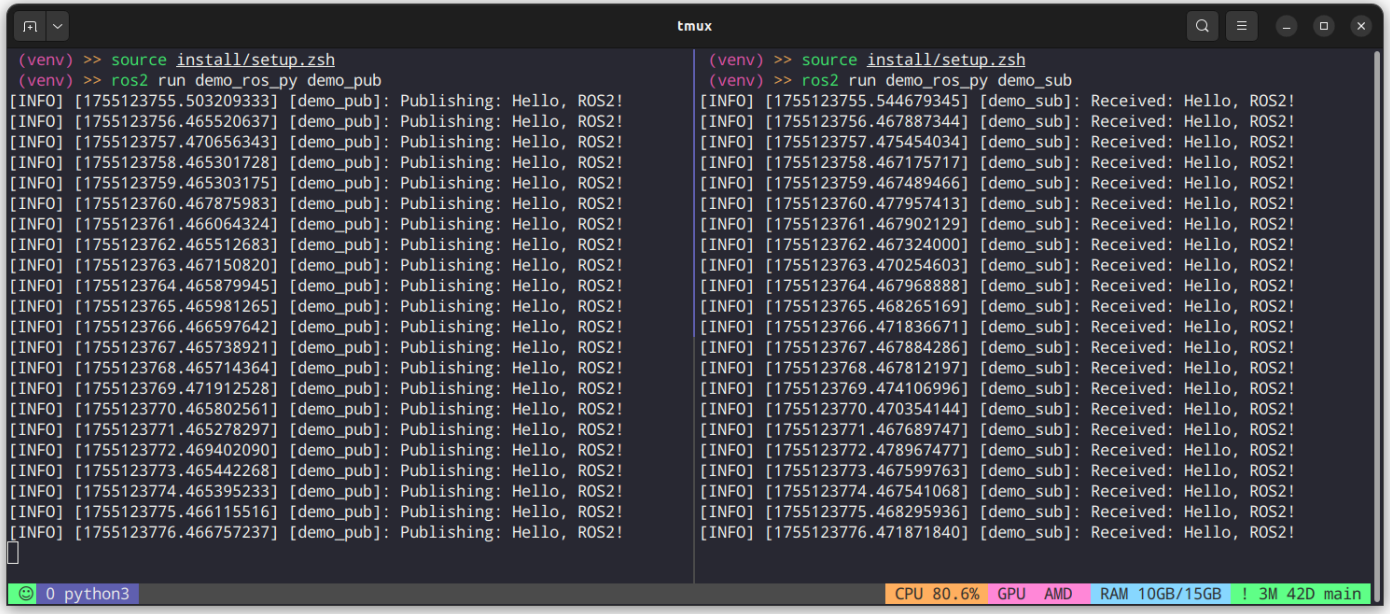
test_flake8.py Runs `flake8` linting

test_pep257.py Checks docstring conventions

<package_name>/ The main Python module directory where we'll write our actual **Python** code

__init__.py Makes this directory a Python package

2.3 Build mechanism



ros2_ws/src/demo_ros_py

Task 6: Mini Code (Python, minimal)

7. Fill in the blanks to create a simple **ROS 2** publisher node in Python:

Python

```python
import rclpy
from rclpy.node import ___
from std_msgs.msg import ___

class MinimalPublisher(___):
    def __init__(___):
        super().__init__('minimal_publisher')
        self.publisher_ = self.create_publisher(String, '___', 10)
        timer_period = ___  # seconds
        self.timer = self.create_timer(timer_period, self.___)
```

```
11
12      def ___(___):
13          msg = ___()
14          msg.data = '___'
15          self.publisher_.___(___)
16          self.get_logger().info(f'Publishing: "{msg.data}"')
17
18  def main(args=None):
19      rclpy.init(args=args)
20      ___ = MinimalPublisher()
21      rclpy.spin(___)
22      ___.destroy_node()
23      rclpy.shutdown()
```

```
24
25 if __name__ == '__main__':
26     main()
```

3. Custom Interface Development

ROS 2 supports three primary types of interfaces for inter-node communication: messages, services, and actions. Each serves a unique purpose in the publish-subscribe or client-server paradigms.

- **Publish-Subscribe** communication
- One-way data transmission
- Asynchronous communication
- Best for: sensor data, status updates, continuous streams

```yaml
# Name: "geometry_msgs/msg/Twist.msg"
geometry_msgs/Vector3 linear
geometry_msgs/Vector3 angular
```

3.3 Services (srv)

- **Request-Response** communication
- Synchronous, blocking calls
- Best for: commands, queries, configuration

```yaml
# Name: "turtlesim/srv/Spawn.srv"
float64 x
float64 y
float64 theta
string name
---
string name
```

3.4 Actions (action)

- **Long-running tasks** with feedback
- Asynchronous with progress updates
- Can be cancelled
- Best for: navigation, manipulation, complex behaviors

```yaml
# Name: "turtlesim/action/RotateAbsolute.action"
float32 theta
---
float32 delta
---
float32 remaining
```

3.6 Basic Interface Commands

```bash
ros2 interface <command>
```

```
Commands:
  list       List all interface types available
  package    Output a list of available interface types within one package
  packages   Output a list of packages that provide interfaces
  proto      Output an interface prototype
  show       Output the interface definition
```

```bash
ros2 interface package turtlesim
```

```
turtlesim/msg/Color
turtlesim/srv/Spawn
turtlesim/msg/Pose
turtlesim/srv/SetPen
turtlesim/srv/Kill
turtlesim/srv/TeleportAbsolute
turtlesim/action/RotateAbsolute
turtlesim/srv/TeleportRelative
```

3.8 Package Structure for Custom Interfaces

```
my_interfaces/
├── CMakeLists.txt
├── package.xml
├── msg/
│   └── SensorData.msg
├── srv/
│   └── GetData.srv
└── action/
    └── NavigateTo.action
```

Key Requirements:

- Separate directories for each interface type
- Proper CMakeLists.txt configuration
- Correct package.xml dependencies

3.9 Message Definition (.msg)

YAML

```yaml
# Name: "my_interfaces/msg/SensorData.msg"
# Header with timestamp and frame
std_msgs/Header header
# Sensor readings
float64 temperature
float64 humidity
float64 pressure
# Status information
bool is_valid
string sensor_id
```

3.9 Message Definition (.msg)

Supported Data Types:

- Built-in: `bool`, `int8`, `uint8`, `int16`, `uint16`, `int32`, `uint32`, `int64`, `uint64`, `float32`, `float64`, `string`
- Arrays: `int32[]`, `string[]`
- Other messages: `geometry_msgs/Point`

3.10 Service Definition (.srv)

```yaml
# Name: "my_interfaces/srv/GetData.srv"
# Request
string sensor_id
float64 timeout_seconds
---
# Response
bool success
string message
```

3.10 Service Definition (.srv)

Structure:

- Above `---`: Request fields
- Below `---`: Response fields
- Can use custom message types

3.11 Action Definition (.action)

```yaml
# Name: "my_interfaces/action/NavigateTo.action")
# Goal
geometry_msgs/Point target_position
float64 max_velocity


---
# Result
bool success
string message
float64 final_distance

---
```

```
14 # Feedback
15 geometry_msgs/Point current_position
16 float64 progress_percentage
17 float64 remaining_distance
```

3.11 Action Definition (.action)

Three Parts:

- **Goal**: What to accomplish
- **Result**: Final outcome
- **Feedback**: Progress updates

3.13 Essential CMakeLists.txt Setup

CMake

```
# Find dependencies
find_package(rosidl_default_generators REQUIRED)
find_package(builtin_interfaces REQUIRED)
find_package(geometry_msgs REQUIRED)
find_package(std_msgs REQUIRED)

# Generate interfaces
rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/SensorData.msg"
  "srv/GetData.srv"
  "action/NavigateTo.action"
  DEPENDENCIES builtin_interfaces geometry_msgs std_msgs
  )
```

3.14 Package.xml Dependencies

```xml
<name>my_interfaces</name>
<version>0.1.0</version>
<description>Custom ROS 2 interfaces</description>

<depend>builtin_interfaces</depend>
<depend>geometry_msgs</depend>
<depend>std_msgs</depend>
```

3.16 Building the Package

```bash
# Build the interface package
colcon build --packages-select my_interfaces

# Source the workspace
source install/setup.bash
```

ros2_ws/src/my_interfaces

Verification:

```bash
# List available interfaces
ros2 interface list | grep my_interface_package

# Show interface definition
ros2 interface show my_interfaces/msg/SensorData

# Check package interfaces
ros2 interface package my_interfaces
```

3.17 Using Custom Interfaces in Python and C++

Python

```python
from my_interfaces.msg import SensorData
from my_interfaces.srv import GetData
from my_interfaces.action import NavigateTo
```

ros2_ws/src/demo_interfaces_py

C++

```cpp
#include "my_interfaces/msg/sensor_data.hpp"
#include "my_interfaces/srv/get_data.hpp"
#include "my_interfaces/action/navigate_to.hpp"
```

ros2_ws/src/demo_interfaces_cpp

3.17 Using Custom Interfaces in Python and C++

Python

```python
# Publisher
publisher = node.create_publisher(SensorData, 'sensor_data', 10)
```

C++

```cpp
// Publisher
auto publisher = node->create_publisher<my_interfaces::msg::SensorData>(
  "sensor_data", 10);
```

3.17 Using Custom Interfaces in Python and C++

Python

```python
# Service client
client = node.create_client(GetData, 'get_data')
```

C++

```cpp
// Service client
auto client = node->create_client<my_interfaces::srv::GetData>(
  "get_data");
```

3.17 Using Custom Interfaces in Python and C++

Python

```python
# Action client
action_client = ActionClient(node, NavigateTo, 'navigate_to')
```

C++

```cpp
// Action client
auto action_client =
rclcpp_action::create_client<my_interfaces::action::NavigateTo>(
  node, "navigate_to");
```

3.19 Interface Validation

Python

```python
# Validate message before publishing
from rclpy.qos import QoSProfile, ReliabilityPolicy

# Set up validation
qos = QoSProfile(
  reliability=ReliabilityPolicy.RELIABLE,
  depth=10
)

# Use with validation
publisher = node.create_publisher(
  SensorData, 'sensor_data', qos_profile=qos
)
```

3.20 Custom Interface Testing

```bash
# Test interface compilation
colcon build --packages-select my_interfaces

# Test interface usage
ros2 run my_interfaces test_interfaces

# Validate with rosbag
ros2 bag record /sensor_data
ros2 bag play recorded_bag
```

3.21 Summary

Interface Types

- Messages: One-way, asynchronous
- Services: Request-response, synchronous
- Actions: Long-running with feedback

Development Process

1. Define interface files (.msg, .srv, .action)
2. Configure CMakeLists.txt and package.xml
3. Build with colcon

Best Practices

- Design for reusability
- Maintain backward compatibility
- Test thoroughly
- Document clearly

Thank you for your attention

Bibliography

Bibliography

Brito, B., Marques, H., Oso, A., Camacho, A., Olivares-Alarcos, A., Foix, S., Perera, A., Porzi, L., Santos, C., & Kappler, D. (2021). ROS2Learn: A reinforcement learning framework for ROS 2. *Journal of Intelligent & Robotic Systems*, *102*(4), 1–19.

Castelló, J., Macenski, S., & Martín, F. (2021). Navigation2: A new generation of navigation for robots using ROS 2. *IEEE Robotics & Automation Magazine*, *28*(4), 87–98.

Di Nardo, D., Cicconetti, C., Giambene, G., Muhammad, T., Spada, F., & Bechini, A. (2022). Security challenges in Robot Operating System 2 (ROS 2): An empirical analysis. *2022 IEEE 23rd International Symposium on a World of Wireless, Mobile and Multimedia Networks (Wowmom)*, 466–472.

Hernández, E., Pérez, V., Martín, F., & Fernández, J. (2021). ROS 2 for robotics applications: A comprehensive review. *Sensors*, *21*(24), 8240.

Joseph, L., & Cacace, J. (2018). *Mastering ROS for Robotics Programming: Design, build, and simulate complex robots using the Robot Operating System*. Packt Publishing Ltd.

Bibliography

Koubaa, A. (2018). Service-oriented Robot Operating System for distributed robotics: A case study. *Robotics and Autonomous Systems*, *108*, 91–109.

Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W. (2022). Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, *7*(66), eabm6074.

Maruyama, Y., Kato, S., & Azumi, T. (2016). Exploring the performance of ROS2. *Proceedings of the 13th International Conference on Embedded Software*, 1–10.

Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R., & Ng, A. Y. (2009). ROS: an open-source Robot Operating System. *ICRA Workshop on Open Source Software*, *3*(3.2), 5.

Reke, M., Peter, D., Schulte-Tigges, J., Schiffer, S., Ferrein, A., Walter, T., & Matheis, D. (2020). Real-time ROS 2 communication for cooperative automated vehicles. *2020 IEEE Intelligent Vehicles Symposium (IV)*, 958–963.